TechPulse

Technical Reference for Assessors

KCA University - Bachelor of Science in Data Science

Research documentation | 1 November 2026

A practical guide to the current research prototype, its evidence and its limits.

Contents

1. Executive summary and evidence boundary . . . 3
2. Architecture . . . 3
3. Data sources and lineage . . . 3
4. Database schema and SQL contract . . . 4
5. Twelve analytical queries . . . 4
6. ETL and quality controls . . . 9
7. Feature engineering and target labels . . . 9
8. Model training and validation . . . 9
9. Saved predictions and explainability . . . 10
10. Streamlit architecture . . . 10
13. Incremental live API observation layer . . . 10
11. Testing and reproducibility . . . 11
12. Limitations and research recommendations . . . 11

Implementation audit for assessors Repository: `C:\Users\Administrator\OneDrive\GitHub\projects\TechPulse` Review date: 1 November 2026

1. Executive summary and evidence boundary

TechPulse is a research prototype with a Streamlit multipage interface, CSV development fallback, PostgreSQL ingestion path, feature engineering, future-activity labels, four candidate classifiers, persisted prediction and explanation artifacts, twelve descriptive SQL queries, and an additive public API observation store. The app has nine guided pages and reads saved outputs; it does not perform on-demand inference.

The current checkout has material reproducibility gaps. The current feature matrix has 53 rows (51 Declining, 2 Stable, 0 Growing); `labelling_summary.json` reports 36 rows (12 per class); saved predictions contain 53 rows (52 predicted Declining, 1 predicted Growing); the model comparison CSV reports perfect scores for four candidates but contains no baseline row; and `evaluation_summary.json` is absent. A serialized KNN artifact from the metric run contains 36 labelled rows total, including an 8-row test split (Growing 2, Stable 3, Declining 3). Therefore those metrics are not a current-run estimate for the 53-row feature table. The UI displays recorded values with explicit warnings; it does not silently rewrite or suppress them.

2. Architecture

```
Observed local CSVs / PostgreSQL tables
                |
                v
src/data_ingestion.py -- schema checks, local derivations, provenance manifest
                |
                v
src/feature_engineering.py -- seven predictors + future-window target columns
                |
                v
src/labelling.py -- Growing / Stable / Declining labels
                |
                v
src/model_training.py -- imputation, stratified random 80/20 split, CV, four estimators
                |
                v
src/evaluation.py -- aggregate comparison, confusion images, baselines when run
                |
                v
outputs/ -- metrics, saved technology rows, SHAP and provenance artifacts
                |
                v
dashboard/ -- nine pages; browse saved artifacts, source trends and supplementary live observations
```

The documented labels, feature design, ingestion, trained artifact and UI are separate lifecycle stages. `pipeline/run_pipeline.py` orchestrates the local run. The dashboard pages use `dashboard/data.py` for defensive loading and shared definitions. The database path requires `DATABASE_URL`; if it is configured but unreachable, ingestion raises a connection error rather than hiding the failure with CSV fallback.

3. Data sources and lineage

The ingestion code defines six warehouse tables:

Table	Intended role	Current local status
`so_questions`	Technology/tag questions, dates, answer and unanswered measures	Mapped from `data/stackexchange.csv` (45,238 rows, 53 tags, 2016-01-01 to 2018-09-25). Extraction provenance still needs independent confirmation.
`dev_sentiment`	Technology-year survey or sentiment features	No `developer_survey_usage.csv` is present in this checkout. The local fallback derives a satisfaction-like proxy from unanswered share and marks `source_type=demo_proxy_from_unanswered_rate`.

Table	Intended role	Current local status
`fortune500_stacks`	Technology/company adoption relations	Deterministic local development rows; not actual technology-use evidence.
`company_profiles`	Company names, sectors and profile fields	`data/fortune.csv` contains 500 company profiles; profile presence does not establish software usage.
`tech_metadata`	Name and category mappings	Category inferred from technology-name rules in local mode.
`question_company_mapping`	Question/company/technology relation	Generated development mapping; not observed provenance.

Survey aggregates in other project versions are based on annual Stack Overflow Developer Survey answers. Such responses are self-selected, vary in question coverage by year, and are not direct enterprise-adoption measures. This checkout has no `scripts/acquire_public_data.py` and no survey aggregate CSV. The README and this reference do not treat absent data as present.

Additional files: `data/ev311.csv` (36,431 rows of civic service-request fields, not used in the predictive pipeline); `data/files.csv` (empty); `data/fortune.csv`; `data/stackexchange.csv`; `data/feature_matrix.csv`. Feature and prediction tables are derived artifacts, not independent raw datasets.

4. Database schema and SQL contract

`docs/schema_diagram.md` describes a descriptive legacy schema with `company`, `fortune500`, `stackoverflow`, `tag_company`, and `tag_type` relationships. The twelve query files reference those names. `src/data_ingestion.py` expects a separate six-table warehouse contract listed above. Before execution in PostgreSQL, reconcile these structures, keys, column names, timestamp semantics, and source lineage. The local CSV fallback preserves pipeline operation but cannot fabricate the missing warehouse evidence.

5. Twelve analytical queries

The following appendix reproduces the repository SQL as checked in. Each query's declared purpose is explanatory; expected outputs depend on the relevant schema and populated tables. No output should be described as research-grade until the source tables and joins are verified.

Query 01: Most active technologies

Research question: Ranks tags by question volume; outputs total questions, unanswered totals, average unanswered share and active dates.

Source file: `queries/01_basic/query1_top_technologies.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Find which technologies generate the most developer questions
SELECT
    tag,
    SUM(question_count) AS total_questions,
    SUM(unanswered_count) AS total_unanswered,
    (AVG(unanswered_pct))::numeric(10,2) AS avg_unanswered_pct,
    COUNT(DISTINCT date) AS days_with_activity
FROM stackoverflow
GROUP BY tag
ORDER BY total_questions DESC
LIMIT 10;
```

Query 02: Unanswered question share

Research question: Finds tags with high unanswered share after a minimum volume condition.

Source file: `queries/01_basic/query2_tech_difficulty.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Hardest technologies based on unanswered question percentage
SELECT
    tag,
    SUM(question_count) AS total_questions,
    SUM(unanswered_count) AS unanswered_count,
    (100.0 * SUM(unanswered_count) / NULLIF(SUM(question_count), 0))::numeric(10,2) AS
    unanswered_percentage
FROM stackoverflow
GROUP BY tag
HAVING SUM(question_count) >= 100
ORDER BY unanswered_percentage DESC
LIMIT 10;
```

Query 03: Monthly activity trend

Research question: Returns monthly questions, unanswered measures and a three-month lag for trend comparison.

Source file: `queries/01_basic/query3_monthly_trends.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Monthly adoption/community trend analysis for all observed technologies.
SELECT
    tag,
    DATE_TRUNC('month', date) AS month,
    SUM(question_count) AS monthly_questions,
    SUM(unanswered_count) AS monthly_unanswered,
    (100.0 * SUM(unanswered_count) / NULLIF(SUM(question_count), 0))::numeric(10,2) AS
    unanswered_percentage,
    LAG(SUM(question_count), 3) OVER (
        PARTITION BY tag
        ORDER BY DATE_TRUNC('month', date)
    ) AS questions_three_months_prior
FROM stackoverflow
GROUP BY tag, DATE_TRUNC('month', date)
ORDER BY tag, month;
```

Query 04: Company technology breadth

Research question: Counts distinct tags linked to each company and orders companies by portfolio breadth.

Source file: `queries/02_intermediate/query4_companies_most_tags.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Find companies that own/manage the most technologies
SELECT
    c.name AS company_name,
    c.ticker,
    COUNT(DISTINCT tc.tag) AS number_of_technologies
FROM company c
JOIN tag_company tc ON c.id = tc.company_id
GROUP BY c.id, c.name, c.ticker
ORDER BY number_of_technologies DESC
LIMIT 10;
```

Query 05: Category-level activity

Research question: Summarizes volume, unanswered share and variation by technology category.

Source file: `queries/02_intermediate/query5_tech_categories.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Compare difficulty across different technology types
SELECT
    tt.type AS technology_category,
    COUNT(DISTINCT tt.tag) AS num_technologies,
    SUM(s.question_count) AS total_questions,
    (AVG(s.unanswered_pct))::numeric(10,2) AS avg_unanswered_pct,
    (STDDEV(s.unanswered_pct))::numeric(10,2) AS difficulty_variance
FROM tag_type tt
JOIN stackoverflow s ON tt.tag = s.tag
```

```
GROUP BY tt.type
ORDER BY total_questions DESC;
```

Query 06: Hardest technology per company

Research question: Ranks the highest-friction technology for each company using question volume and unanswered share.

Source file: `queries/02_intermediate/query6_hardest_per_company.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- For each company, find their hardest technology
WITH company_tech_difficulty AS (
    SELECT
        c.name AS company_name,
        tc.tag,
        SUM(s.question_count) AS total_questions,
        (100.0 * SUM(s.unanswered_count) / NULLIF(SUM(s.question_count), 0))::numeric(10,2) AS
    unanswered_pct
    FROM company c
    JOIN tag_company tc ON c.id = tc.company_id
    JOIN stackoverflow s ON tc.tag = s.tag
    GROUP BY c.name, tc.tag
    HAVING SUM(s.question_count) >= 50
),
ranked_difficulty AS (
    SELECT
        company_name,
        tag,
        unanswered_pct,
        ROW_NUMBER() OVER (PARTITION BY company_name ORDER BY unanswered_pct DESC) AS difficulty_rank
    FROM company_tech_difficulty
)
SELECT
    company_name,
    tag AS hardest_technology,
    unanswered_pct AS unanswered_percentage
FROM ranked_difficulty
WHERE difficulty_rank = 1
ORDER BY unanswered_pct DESC
LIMIT 15;
```

Query 07: Growth momentum

Research question: Compares a recent 30-day window with earlier activity windows.

Source file: `queries/02_intermediate/query7_growth_momentum.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Growth momentum using the latest 30-day window versus the previous 30-day window.
WITH date_range AS (
    SELECT
        MIN(date) as earliest,
        MAX(date) as latest
    FROM stackoverflow
),
tech_comparison AS (
    SELECT
        s.tag,
        SUM(CASE WHEN s.date >= (SELECT latest - INTERVAL '30 days' FROM date_range)
                 THEN s.question_count ELSE 0 END) AS recent_questions,
        SUM(CASE WHEN s.date < (SELECT latest - INTERVAL '30 days' FROM date_range)
                  AND s.date >= (SELECT latest - INTERVAL '60 days' FROM date_range)
                 THEN s.question_count ELSE 0 END) AS past_questions
    FROM stackoverflow s
    CROSS JOIN date_range
    GROUP BY s.tag
)
SELECT
    tag,
    recent_questions,
    past_questions,
    CASE
        WHEN past_questions > 0
        THEN (100.0 * (recent_questions - past_questions) / past_questions)::numeric(10,2)
        ELSE 0
    END AS growth_percentage
```

```
FROM tech_comparison
WHERE recent_questions > 0 OR past_questions > 0
ORDER BY growth_percentage DESC
LIMIT 20;
```

Query 08: Parent-company portfolio

Research question: Compares parent-company and subsidiary technology portfolios and question totals.

Source file: `queries/03_advanced/query8_parent_company_analysis.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Compare parent companies vs their subsidiaries' tech portfolios
WITH company_techs AS (
    SELECT
        c.id,
        c.name,
        c.parent_id,
        COUNT(DISTINCT tc.tag) AS tech_count,
        SUM(s.question_count) AS total_developer_questions
    FROM company c
    LEFT JOIN tag_company tc ON c.id = tc.company_id
    LEFT JOIN stackoverflow s ON tc.tag = s.tag
    GROUP BY c.id, c.name, c.parent_id
)
SELECT
    parent.name AS parent_company,
    COUNT(child.id) AS num_subsidiaries,
    SUM(child.tech_count) AS total_technologies_across_subsidiaries,
    SUM(child.total_developer_questions) AS total_developer_activity,
    parent.tech_count AS parent_tech_count,
    parent.total_developer_questions AS parent_developer_activity
FROM company_techs parent
JOIN company_techs child ON parent.id = child.parent_id
WHERE parent.parent_id IS NULL
GROUP BY parent.name, parent.tech_count, parent.total_developer_questions
ORDER BY total_developer_activity DESC
LIMIT 10;
```

Query 09: Technology health composite

Research question: Builds a descriptive health score from activity, unanswered share and active-day coverage.

Source file: `queries/03_advanced/query9_tech_health_score.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Identify technologies at risk of decline based on multiple factors
WITH tech_health AS (
    SELECT
        tag,
        SUM(question_count) AS total_questions,
        ROUND(AVG(unanswered_pct)::numeric, 2) AS avg_difficulty,
        COUNT(DISTINCT date) AS active_days,
        MAX(date) AS last_activity,
        ROUND((SUM(question_count) * (1 - AVG(unanswered_pct) / 100))::numeric, 2) AS health_score
    FROM stackoverflow
    GROUP BY tag
    HAVING SUM(question_count) > 500
)
SELECT
    tag,
    total_questions,
    avg_difficulty         AS difficulty_pct,
    active_days,
    last_activity,
    health_score,
    CASE
        WHEN last_activity < CURRENT_DATE - INTERVAL '90 days' THEN 'CRITICAL - No recent activity'
        WHEN avg_difficulty > 40                                THEN 'HIGH - Too difficult'
        WHEN health_score < 1000                               THEN 'MEDIUM - Low engagement'
        ELSE 'HEALTHY'
    END AS risk_status
FROM tech_health
ORDER BY
    CASE
        WHEN last_activity < CURRENT_DATE - INTERVAL '90 days' THEN 1
        WHEN avg_difficulty > 40                                THEN 2
```

```
        WHEN health_score < 1000                                THEN 3
        ELSE 4
    END,
    health_score ASC
LIMIT 20;
```

Query 10: Question quality

Research question: Summarizes question volume, unanswered share and quality groups by tag.

Source file: `queries/03_advanced/query10_question_quality.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Analyze the relationship between question volume and answer quality
SELECT
    tag,
    SUM(question_count) AS total_questions,
    (AVG(question_pct))::numeric(10,2) AS avg_question_share,
    (AVG(unanswered_pct))::numeric(10,2) AS avg_unanswered_rate,
    (100.0 * SUM(unanswered_count) / NULLIF(SUM(question_count), 0))::numeric(10,2) AS
    overall_unanswered_rate,
    CASE
        WHEN SUM(question_count) > 10000 AND (100.0 * SUM(unanswered_count) /
    NULLIF(SUM(question_count), 0)) < 20 THEN 'High Volume, High Quality'
        WHEN SUM(question_count) > 10000 AND (100.0 * SUM(unanswered_count) /
    NULLIF(SUM(question_count), 0)) > 30 THEN 'High Volume, Poor Quality'
        WHEN SUM(question_count) < 1000 AND (100.0 * SUM(unanswered_count) /
    NULLIF(SUM(question_count), 0)) < 20 THEN 'Low Volume, High Quality'
        WHEN SUM(question_count) < 1000 AND (100.0 * SUM(unanswered_count) /
    NULLIF(SUM(question_count), 0)) > 30 THEN 'Low Volume, Poor Quality'
        ELSE 'Average'
    END AS quality_category
FROM stackoverflow
GROUP BY tag
HAVING SUM(question_count) >= 500
ORDER BY total_questions DESC
LIMIT 20;
```

Query 11: Daily activity pattern

Research question: Compares aggregate community activity by weekday.

Source file: `queries/03_advanced/query11_daily_patterns.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Analyze which days of week have highest developer activity
SELECT
    EXTRACT(DOW FROM date) AS day_of_week,
    CASE EXTRACT(DOW FROM date)
        WHEN 0 THEN 'Sunday'
        WHEN 1 THEN 'Monday'
        WHEN 2 THEN 'Tuesday'
        WHEN 3 THEN 'Wednesday'
        WHEN 4 THEN 'Thursday'
        WHEN 5 THEN 'Friday'
        WHEN 6 THEN 'Saturday'
    END AS day_name,
    AVG(question_count) AS avg_daily_questions,
    SUM(question_count) AS total_questions,
    (AVG(unanswered_pct))::numeric(10,2) AS avg_unanswered_pct
FROM stackoverflow
GROUP BY EXTRACT(DOW FROM date)
ORDER BY avg_daily_questions DESC;
```

Query 12: Company diversity

Research question: Uses technology category breadth to describe company portfolio diversity.

Source file: `queries/03_advanced/query12_company_diversity.sql`

Expected output: The query's selected aliases and records, depending on the source schema and filters. This is descriptive output, not a prediction.

```
-- Calculate technology diversity score for each company
SELECT
    c.name AS company_name,
```

```
    COUNT(DISTINCT tc.tag) AS num_technologies,
    COUNT(DISTINCT tt.type) AS num_tech_categories,
    SUM(s.question_count) AS total_developer_questions,
    ROUND((SUM(s.question_count) * 1.0 / NULLIF(COUNT(DISTINCT tc.tag), 0))::numeric, 2) AS
    avg_questions_per_tech,
    CASE
        WHEN COUNT(DISTINCT tt.type) >= 3 THEN 'High Diversity'
        WHEN COUNT(DISTINCT tt.type) >= 2 THEN 'Medium Diversity'
        ELSE 'Low Diversity'
    END AS diversity_rating
FROM company c
JOIN tag_company tc ON c.id = tc.company_id
LEFT JOIN tag_type tt ON tc.tag = tt.tag
LEFT JOIN stackoverflow s ON tc.tag = s.tag
GROUP BY c.id, c.name
ORDER BY total_developer_questions DESC
LIMIT 10;
```

6. ETL and quality controls

`src/data_ingestion.py` loads six PostgreSQL tables if `DATABASE_URL` exists. It checks required table presence and computes row count, missingness, date ranges, technology counts, and null-threshold warnings. In local mode it reads Stack Exchange and Fortune CSVs, derives the remaining tables deterministically and writes a data quality report and source-mode manifest. Connection errors and missing tables are surfaced.

`src/technology_normalization.py` standardizes technology names. Feature engineering parses/aggregates community activity, computes derived signals and target-window fields, and saves `data/feature_matrix.csv`. The current data files are small demonstration matrices; report the local source mode and proxy fields with any UI result.

7. Feature engineering and target labels

The schema lists seven model predictors:

1. `technology_health_score`
2. `growth_momentum_index`
3. `question_quality_score`
4. `company_diversity_score`
5. `sentiment_delta`
6. `adoption_velocity`
7. `community_decay_rate`

The target compares future-window average monthly question volume with recent average monthly activity. Ratio >= 1.20 maps to Growing; <= 0.80 maps to Declining; the interval between maps to Stable. Future columns (`future_volume`, `future_avg_monthly_volume`, `recent_avg_monthly_volume`, `future_growth_ratio`) define the outcome and must not leak into predictors.

Current feature matrix counts and saved label summary conflict. The dashboard flags this and displays target labels separately from predicted labels. Do not regenerate the label summary independently from the feature matrix without investigating the source mismatch.

8. Model training and validation

`src/model_training.py` implements Logistic Regression, K-Nearest Neighbours, Random Forest and XGBoost, using imputation and weighted F1 for model selection. It uses `train_test_split(test_size=0.2, stratify=y, random_state=42)`, followed by stratified cross-validation for tuning where class support allows. This is a random stratified split, not a temporal holdout or grouped-by-technology test. Repeated observations from the same technology or adjacent periods could occur in both partitions.

`src/evaluation.py` writes Accuracy, Weighted F1, Precision, Recall and Macro ROC-AUC when its conditions allow, confusion matrices, and majority/momentum baselines during a fresh evaluation. The current checked-in comparison CSV has four candidate rows, all with score 1.0, no baseline, and no accompanying `evaluation_summary.json`. The current feature file has 53 records, while the label summary and loaded serialized model correspond to a 36-row run; the model

test target has eight rows. Thus the saved scores cannot be attributed to the current table with confidence. The application keeps them labelled as recorded snapshot metrics and warns against interpretation.

Accuracy alone can be misleading under class imbalance. Weighted F1 gives more influence to common classes; macro F1 gives equal class weight and is not present in the current CSV. Per-class precision/recall and support, confusion matrices and a majority baseline are required to evaluate rare categories meaningfully. ROC-AUC is meaningful only when the test split has appropriate class coverage and valid probability outputs.

9. Saved predictions and explainability

`outputs/technology_predictions.csv` contains 53 technology rows, target labels, predicted labels, three class scores, confidence fields, and a saved risk score. In the current file, 52 predicted rows are Declining and one Growing. Target labels have 51 Declining and two Stable. The distinction is important: targets derive from labels; model predictions derive from the saved estimator.

Risk is `(1 - P(Growing)) * 100`; it is not a calibrated or separately validated probability of decline/failure. Confidence is maximum class score and is not itself a guarantee of correctness. Global and local SHAP artifacts attribute model output across features; attribution is not causal evidence. Artifact age/version and feature alignment should be checked before presenting a SHAP chart as the explanation of the current selected model.

10. Streamlit architecture

`dashboard/app.py` registers nine pages. Shared visual components in `dashboard/components/ui.py` implement the linen `#E8DFD2`, taupe `#8B7D6B`, and charcoal `#3A3734` theme. `dashboard/data.py` loads cached CSV/JSON artifacts, separates target from prediction labels, and computes the risk indicator only from a stored Growing probability. Other pages read actual CSV community observations; absent matched history is not rendered as zero. The interface includes a Development Data badge, persistent help links, plain-language explanations and a research disclaimer.

Pages:

- Start Here: purpose, active mode, current artifact warnings, target/predicted distributions.
- Technology Explorer: saved row search and matching community time series.
- Technology Rankings: saved snapshot filters, table, chart and CSV export.
- Community Health: question totals, unanswered share, monthly volume, survey shown separately only if present.
- Prediction Explorer: saved class/probabilities/signals and matching SHAP attribution where available.
- Model Performance & Validation: recorded metrics, class support, baseline/split caveats.
- Methodology & Data: provenance, table contract, label details, and collapsible actual query sources.
- About & User Guide: FAQ, glossary and responsible use.
- Live Technology Intelligence: timestamped, separate public API observations, source status, sampled-data caveats and no page-load API calls.

No new-input inference or model training is triggered by a selector.

13. Incremental live API observation layer

The four collectors in `src/live_data/collectors.py` use documented public interfaces: GitHub REST repository search, Stack Exchange tag information and paginated questions, the PyPI project JSON API, the npm registry package document and npm's monthly downloads endpoint. Requests have finite timeouts, bounded exponential retries, retry-after and GitHub rate-limit reset handling, JSON shape checks, logging and source-isolated errors. Stack Exchange `backoff` responses are honoured. `GITHUB_TOKEN` and `STACKEXCHANGE_KEY` are optional environment credentials; the anonymous public APIs remain usable within their lower quotas.

GitHub queries search repository names and descriptions and collect up to the configured page cap (`TECHPULSE_API_MAX_PAGES`, maximum 100 items per page). Repository counts are explicitly distinguished as query matches versus sampled repository metrics. Stack Exchange reads the current tag question count and at most the configured number of pages from the latest 30 days; recent question aggregates are labelled sampled, and individual

question observations keep the API question identifier and creation timestamp. PyPI release dates are recovered from `upload_time_iso_8601` or `upload_time`; the endpoint's `downloads` value is ignored because PyPI documents it as deprecated and always `-1`. npm uses its public package metadata `time` object and its last-month downloads API. Registries are queried only for verified package associations in `data/live_package_map.json`; no package name is guessed from a technology tag.

Observation schema and lifecycle

The additive PostgreSQL table `techpulse_live_observations` is created on first write. It contains technology identifier/name, source, metric name, numeric or text value, source observation timestamp, collection timestamp, source-specific identifier and JSON metadata. Its primary key is `(technology_id, source, metric_name, source_identifier, observed_at)` and inserts use `ON CONFLICT DO NOTHING`, preserving existing observations. When `DATABASE_URL` is unset, the same normalized rows are appended to `outputs/live_data/live_observations.csv` with duplicate identities removed through a temporary-file replace. The original six-table data-ingestion contract and its CSV fallback are unchanged. The dashboard reads live rows through this same storage module; when the database URL is configured, database errors are surfaced rather than silently switching to CSV.

The independent scheduler in `scripts/schedule_live_data.py` runs daily by default, and can be kept alive by a service manager or Windows Task Scheduler. It selects five high-volume technologies from the existing catalogue unless `TECHPULSE_LIVE_TECHNOLOGIES` is specified. All four sources refresh at that interval; package releases and download aggregates are refreshed daily. The UI never performs API calls during page loading. Its refresh button starts `scripts/collect_live_data.py` as a background process for only the selected technology. Per-source status and errors are recorded under ignored runtime files in `outputs/live_data/`.

Research boundary

Live observations are descriptive and remain outside `src/feature_engineering.py`, labelling, model training and saved historical evaluation. GitHub repository sample measurements are not enterprise adoption or production use. Stack Exchange question volume measures site-specific support demand, not adoption. Registry metadata and downloads describe packages, not unique developers or production deployment. Historical release dates are source facts; current stars, tag counts and download counts are snapshots collected now and must not be back-projected into earlier periods. Any future modelling use requires a separate as-of-time feature study, temporal leakage review, source comparability and coverage audit, bias/sufficiency analysis, and new evaluation with documented baselines.

11. Testing and reproducibility

Tests are run with `python -m pytest tests/ -q`. Dashboard pages are also exercised with Streamlit `AppTest`; browser screenshots are captured from the actual running app with `python scripts/capture_redesign_screenshots.py`. Live-source test results and capture notes are in `outputs/live_data_test_report.md`; the earlier eight-page redesign report remains a historical record in `outputs/redesign_test_report.md`.

For a reproducible run, record Python/package versions, source mode, input checksums and timestamps, model artifact identifier, feature table row count, target distributions, random seed, split membership, split class counts, metrics, and baseline results in a run-specific directory. Do not overwrite prior run metadata or model artifacts during a research comparison. Never commit database secrets.

12. Limitations and research recommendations

- Local company adoption, mapping and category inputs are derived proxies.
- There is no survey aggregate CSV in this checkout; the fallback satisfaction-like signal is derived from community unanswered data, not survey sentiment.
- The model comparison and label summary do not correspond to the current 53-row feature file.
- Candidate metrics are perfect in the current CSV but not independently auditable; baseline rows and run-level split metadata are absent.
- The split used in source is random stratified, not time-forward or group-aware.
- Current label support is highly imbalanced and has no Growing records in the feature matrix.

- Time alignment across source records is not documented consistently; survey years, where obtained externally, would be later than the 2016-2018 local community observations.
- The interface reads saved predictions; user-input inference is not implemented.
- The twelve SQL files reference a schema different from the six-table ingestion contract.

Recommended research sequence: reconcile ingestion and SQL schemas; verify public-source provenance and licensing; replace development proxy tables with time-aligned observed evidence; create versioned feature/label runs; adopt grouped and chronological validation appropriate to the target; compare with majority and domain baselines; report per-class precision/recall/F1 and calibration; preserve run-level metadata; then update the UI and claims from those verified outputs.